Máy tìm kiếm cho hust.edu.vn

Kỹ thuật của dịch vụ search: chỉ mục, xếp hạng, PageRank, lọc rác và trùng

Đồ án môn Tìm kiếm thông tin (IT4863)

Năm 2026

Tổng quan dịch vụ search

- **Đầu vào:** dữ liệu crawler lưu trong MongoDB: `html` (trang web), `other` (pdf, docx...), `picture` (ảnh), `graph` (cạnh liên kết `source` → `destinations`).
- **Công nghệ:** Java 21, Apache Lucene 10, HTTP API `/search`, `/reindex`, `/ping`.

Dựng chỉ mục (`IndexBuilder`)
1. PageRank trên `graph`
2. Độ lạ, ngưỡng tự động: đánh dấu trang rác
3. SHA-1 nội dung: đánh dấu trang trùng
4. Ghi tài liệu Lucene nhiều trường

chỉ mục →

Truy vấn (`Searcher`)
1. Truy hồi: BM25 nhiều trường, lấy top 100
2. Chấm lại: cụm từ, PageRank, độ mới
3. Lọc rác và trùng (profile D)
4. Phân trang, tạo snippet

- Hai giai đoạn: BM25 rẻ cho toàn bộ kho, các tín hiệu đắt hơn chỉ tính trên 100 kết quả đầu.
- Trang rác và trang trùng vẫn được index, chỉ bị loại lúc truy vấn, nên có thể so sánh có và không lọc trên cùng một chỉ mục.

Quy trình dựng chỉ mục

- Dựng lại toàn bộ (`OpenMode.CREATE`). Duyệt Mongo nhiều lượt, không giữ nội dung trong bộ nhớ.

Lượt	Công việc
0	Đọc `graph`, dựng đồ thị liên kết, tính PageRank
1	Trên các trang html đủ dài: đếm document frequency $\mathrm{df}(w)$ của từng từ
2	Tính độ lạ từng trang, chọn ngưỡng rác tự động; tính khoá SHA-1 nội dung, chọn bản giữ lại cho mỗi nhóm trùng. Chỉ giữ url và số
3	Ghi từng tài liệu Lucene kèm cờ `junk` (`spam` hoặc `dup`)

- `POST /reindex` dựng ở luồng nền, xong thì `SearcherManager.maybeRefresh()`: truy vấn vẫn chạy trong lúc dựng.
- Mọi chuỗi đi qua chuẩn hoá Unicode NFC trước khi index và trước khi truy vấn, vì tiếng Việt có hai cách mã hoá cùng một chữ có dấu.

Phân tích văn bản tiếng Việt

- Không dùng thư viện tách từ. Mỗi âm tiết là một token (`StandardTokenizer`, UAX #29); từ ghép được biểu diễn bằng bigram.
- Giữ cả bản có dấu và bản bỏ dấu, vì người dùng thường gõ không dấu và url, tên file thường không dấu.

Analyzer	Chuỗi xử lý	Ví dụ: Bách Khoa Hà Nội
`vi`	tokenizer, chữ thường	`bách`, `khoa`, `hà`, `nội`
`viBi`	`vi`, rồi bigram (không giữ unigram)	`bách khoa`, `khoa hà`, `hà nội`
`viAscii`	`vi`, rồi `ASCIIFoldingFilter` (cả đ thành d)	`bach`, `khoa`, `ha`, `noi`
`urlTokens`	tách theo ký tự không phải chữ hoặc số, chữ thường, bỏ dấu	`.../hoc-phi-654580.html` thành `hoc`, `phi`, `654580`, `html`

- Truy vấn học phí được biểu diễn bằng hai unigram `học`, `phí` và một bigram `học phí`. Bigram bắt phần lớn từ ghép mà không cần tách từ.

Các trường của chỉ mục và BM25 theo trường

Trường	Loại	Analyzer	Mục đích
`url`, `host`, `collection`, `format`	StringField	không	Định danh, lọc theo host và loại
`title`, `content`	TextField	`vi`	Khớp unigram; `content` lưu vị trí và offset cho phrase query và snippet
`title_bi`, `content_bi`	TextField	`viBi`	Khớp cụm hai âm tiết
`title_ascii`, `content_ascii`	TextField	`viAscii`	Khớp truy vấn không dấu
`url_tokens`	TextField	`urlTokens`	Khớp từ trong url
`sig:pagerank`, `sig:recency`	FeatureField		Tín hiệu ngoài văn bản
`junk`	StringField		`spam` hoặc `dup`

BM25 với tham số riêng từng trường

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{IDF}(t)\, \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1\left(1 - b + b\,\frac{|d|}{\mathrm{avgdl}}\right)}$$

- $k_1 = 1.2$ cho mọi trường. $b = 0.75$ cho `title*` và `url_tokens`; $b = 0.5$ cho `content*`, để tài liệu dài như PDF không bị phạt độ dài quá nặng.

Giai đoạn 1: truy hồi bằng BM25 nhiều trường

- Một `BooleanQuery` gồm các mệnh đề `SHOULD`, mỗi mệnh đề là truy vấn đã phân tích bằng analyzer của trường đó, nhân trọng số. Lấy 100 tài liệu điểm cao nhất.

Trường	Trọng số	Từ profile
`url_tokens`	1.5	A
`title`	3	A
`content`	1	A
`title_bi`	3	B
`content_bi`	1.5	B
`title_ascii`	2	B
`content_ascii`	0.7	B

- Tiêu đề nặng gấp 3 nội dung.
- Bigram trong nội dung nặng hơn unigram vì khớp cụm là bằng chứng mạnh hơn khớp rời.
- Bản bỏ dấu nhẹ hơn bản có dấu, để trang viết đúng dấu xếp trên trang chỉ khớp sau khi bỏ dấu.
- **Truy vấn không dấu** (`lich thi`): chỉ hỏi `title_ascii` (trọng số 3), `content_ascii` (1) và `url_tokens`, để trang có dấu và trang chỉ có tên file không dấu được xét ngang nhau.
- **Bộ lọc** dạng `FILTER`, không tính điểm: `type`, `host`; profile D thêm `MUST_NOT junk`.

Giai đoạn 2: chấm lại trên top 100

- `QueryRescorer`: điểm mới = điểm BM25 + điểm truy vấn phụ (nếu khớp).

Mệnh đề của truy vấn phụ	Trọng số	Ghi chú
Phrase query trên `title`, slop 2	2	Các token đứng gần nhau, đúng thứ tự, cho phép chèn tối đa 2 vị trí. Bỏ qua nếu truy vấn có một token
Phrase query trên `content`, slop 2	1	Như trên
PageRank, `FeatureField` bão hoà	2	Mọi tài liệu có tín hiệu này
Độ mới, `FeatureField` bão hoà	2	Chỉ khi `fresh=1`

Hàm bão hoà và các tín hiệu

$$\text{score} = w \cdot \frac{s}{s + \text{pivot}}, \qquad s_{\text{pr}} = \max(\text{PR} \times N,\, 10^{-4}), \qquad s_{\text{rec}} = \frac{1}{1 + \text{tuổi (ngày)}/30}$$

- Hàm bão hoà tăng nhanh ở giá trị nhỏ rồi phẳng dần, nên trang có PageRank rất lớn không lấn át điểm văn bản. `pivot` do Lucene ước lượng từ phân bố của tín hiệu.
- Nhân PR với số nút N để giá trị trung bình xấp xỉ 1, không phụ thuộc kích thước đồ thị.

PageRank trên đồ thị liên kết

Mô hình random surfer, hệ số $d = 0.85$:

$$\mathrm{PR}(v) = \frac{1-d}{N} + d\left(\sum_{u \to v} \frac{\mathrm{PR}(u)}{\mathrm{outdeg}(u)} + \frac{\mathrm{dangling}}{N}\right)$$

- $\mathrm{dangling}$ là tổng PageRank của các nút không có liên kết ra, chia đều cho mọi nút để tổng PageRank luôn bằng 1.

Dựng đồ thị

- Mỗi url là một nút, đánh chỉ số theo thứ tự gặp; cạnh lưu trong mảng `int`.
- Bỏ cạnh thiếu đầu mút và cạnh tự trỏ.
- Khử cạnh trùng bằng khoá `(a << 32) | b`.

Lặp

- Khởi tạo $1/N$, tối đa 50 vòng, dừng khi tổng sai khác tuyệt đối giữa hai vòng dưới 10^{-6}.
- Độ phức tạp $O(\text{số vòng} \times (N + E))$, chạy trong bộ nhớ.

- PageRank được dùng ở hai chỗ: tín hiệu khi chấm lại, và tiêu chí chọn bản giữ lại trong nhóm trang trùng.

Lọc trang rác bằng độ lạ

Quan sát. Một số trang hust bị chèn nội dung rác (từ khoá quảng cáo). Từ vựng của chúng gần như không trùng với phần còn lại của kho, trong khi trang thật dùng từ vựng chung (đào tạo, sinh viên, thông báo).

Độ đo. Với trang html có ít nhất 50 từ:

$$\text{độ lạ}(p) = \frac{1}{|W(p)|} \sum_{w \in W(p)} \log \frac{N}{\mathrm{df}(w)}$$

- $W(p)$: các chuỗi chữ cái của tiêu đề và nội dung (tính cả từ lặp), chữ thường, NFC. N: số trang html đủ dài. $\mathrm{df}(w)$: số trang chứa từ w.
- Đây là IDF trung bình của trang: từ phổ biến cho giá trị gần 0, từ chỉ xuất hiện ở một trang cho $\log N$.
- Trang dưới 50 từ không đủ để ước lượng nên không bao giờ bị coi là rác. Chỉ xét collection `html`.
- Trang có độ lạ vượt ngưỡng được gắn `junk:spam`. Ngưỡng được chọn tự động ở slide sau.

Chọn ngưỡng rác tự động: đáy thung lũng của histogram

Ý tưởng. Histogram độ lạ có hai cụm: cụm lớn là trang thật, cụm nhỏ nằm cao hơn là trang rác. Ngưỡng là đáy thung lũng giữa hai cụm (phương pháp Prewitt và Mendelsohn). Không đặt ngưỡng tay vì ngưỡng phụ thuộc kho dữ liệu.

Thuật toán

1. Chia độ lạ thành các bin rộng 0.1, đếm số trang mỗi bin: histogram gốc.
2. Làm mượt bằng trung bình ba bin kề nhau, lặp đến khi chỉ còn đúng hai đỉnh (tối đa 200 vòng).
3. Với hai đỉnh $a < b$:
   - Đỉnh phải cao hơn đỉnh trái: không có ngưỡng (cụm phải đông hơn thì không phải rác).
   - Tìm bin thấp nhất v giữa a và b trên histogram gốc, vì làm mượt lâu sẽ lấp mất thung lũng.
   - Nếu $\mathrm{raw}[v] \le 0.5 \cdot h[b]$ thì ngưỡng là $\min + (v + 0.5) \times 0.1$; ngược lại coi là nhiễu và làm mượt tiếp.
4. Còn 0 hoặc 1 đỉnh, hoặc hết 200 vòng: không gắn cờ trang nào.

- Giới hạn: nếu số trang rác quá ít để tạo thành một đỉnh riêng thì không phát hiện được.

Khử trang trùng nội dung

Vấn đề. Trang mirror, bản in, bản đổi ngôn ngữ giống hệt làm kết quả bị lặp.

Khoá nội dung

$$\mathrm{key}(p) = \mathrm{SHA\text{-}1}\big(\text{trim}(\text{gộp khoảng trắng}(\text{chữ thường}(\mathrm{NFC}(\text{content}))))\big)$$

- Nội dung dưới 200 ký tự không có khoá, vì quá ngắn để coi là trùng. Khoá tính cho cả ba collection.

Chọn bản giữ lại trong mỗi nhóm cùng khoá

1. PageRank cao nhất.
2. Hoà thì chọn url ngắn hơn.
3. Trang đã bị đánh dấu rác không được làm bản giữ lại.

- Các url còn lại của nhóm được gắn `junk:dup`. Rác được xét trước, nên trang vừa rác vừa trùng chỉ tính là `spam`.

Bốn profile xếp hạng

- Mỗi profile thêm một tầng kỹ thuật lên profile trước, để đo được đóng góp của từng tầng trên cùng một chỉ mục.

Profile	Truy hồi	Chấm lại	Lọc rác và trùng
A	BM25 trên `url_tokens`, `title`, `content`	không	không
B	A + bigram + bỏ dấu, nhánh truy vấn không dấu	không	không
C	như B	cụm từ + PageRank (+ độ mới)	không
D (mặc định)	như B	như C	có

- Ví dụ truy vấn học phí với profile D: truy hồi trên 7 trường có trọng số, loại rác và trùng, lấy top 100; chấm lại cộng điểm cho trang có cụm học phí liền nhau trong tiêu đề và trang có PageRank cao; trả 10 kết quả đầu.

Phương pháp đánh giá

Dữ liệu. 25 truy vấn trộn có dấu, không dấu và tên đơn vị (ví dụ `học phí 2025`, `lich thi`, `soict tuyển sinh`). Nhãn liên quan do người gán: 0 không liên quan, 1 liên quan một phần, 2 liên quan hoàn toàn.

Pooling. Chạy cả 4 profile cho mỗi truy vấn, gộp top 10 của từng profile để gán nhãn. Mọi tài liệu được bất kỳ profile nào xếp cao đều được người gán nhãn xem, tránh thiên vị một hệ. Nhãn cũ được giữ khi chạy lại.

Độ đo (top $K = 10$, không lọc type và host, `fresh=0`)

$$\mathrm{P@10} = \frac{|\{i \le 10 : g_i \ge 1\}|}{10}, \qquad \mathrm{DCG@10} = \sum_{i=1}^{10} \frac{2^{g_i} - 1}{\log_2(i+1)}, \qquad \mathrm{nDCG@10} = \frac{\mathrm{DCG@10}}{\mathrm{IDCG@10}}$$

- IDCG lấy từ mọi nhãn đã gán của truy vấn, sắp giảm dần. Grade 2 có độ lợi 3, gấp ba grade 1.
- So sánh B với A: đóng góp của bigram và bỏ dấu. C với B: cụm từ và PageRank. D với C: lọc rác và khử trùng.

Snippet và API

Snippet

- Tối đa 2 đoạn từ `content` bằng `UnifiedHighlighter`, từ khớp bọc trong `<b>`.
- Truy vấn không dấu chỉ khớp `content_ascii`: highlight trên trường bỏ dấu rồi dùng lại offset trên văn bản gốc, vì bỏ dấu không đổi vị trí ký tự.
- Không có đoạn khớp thì lấy 200 ký tự đầu của nội dung.

API

Endpoint	Mô tả
`GET /search`	Tham số `q`, `type` (html, other, picture), `host`, `profile` (A đến D), `fresh`, `page`, `size` (tối đa 50). Chỉ 100 kết quả đầu được xếp hạng. Trả `total`, `tookMs`, danh sách `hits` gồm url, title, host, score, snippet
`POST /reindex`	Dựng lại chỉ mục ở nền (202; đang dựng thì 409)
`GET /ping`	Số tài liệu, thời điểm index gần nhất, số trang rác và trùng, ngưỡng độ lạ

Giới hạn kỹ thuật và hướng mở rộng

- **Tách từ:** bigram thay cho tách từ tiếng Việt; cụm đảo thứ tự không khớp bigram. Hướng mở rộng: tách từ bằng pyvi.
- **Chỉ mục:** dựng lại toàn bộ mỗi lần, chưa cập nhật từng phần. Tín hiệu độ mới tính lúc index nên lệch dần nếu không reindex.
- **PageRank:** mọi liên kết có trọng số như nhau, nên liên kết điều hướng (menu, chân trang) làm tăng điểm của trang gốc. Chưa dùng anchor text.
- **Lọc rác:** không phát hiện được nếu số trang rác quá ít để tạo đỉnh riêng trên histogram.
- **Xếp hạng:** chỉ 100 kết quả đầu được chấm lại và phân trang.
- **Chưa làm:** gợi ý khi gõ, sửa chính tả.